EX : 6

Building an anonymous FTP scanner using python.

AIM:

To develop a python based anonymous FTP scanner using the built in ftplib module and determine whether an FTP server permits anonymous access.

Procedure:

1) Install python and import FTP and all error class from python ftplib module.

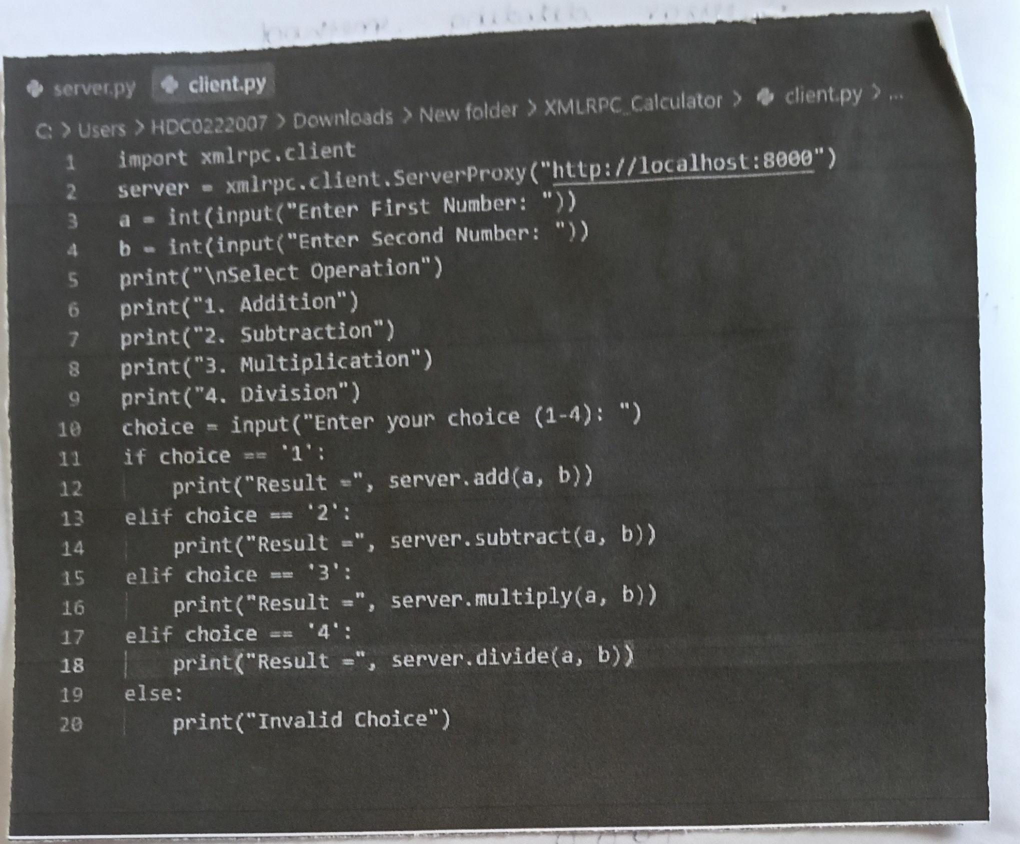
2) Define anonymous-ftp.scan() to perform FTP scan.

3) Enter authorised FTP server address and establish a connection to FTP server using port 2

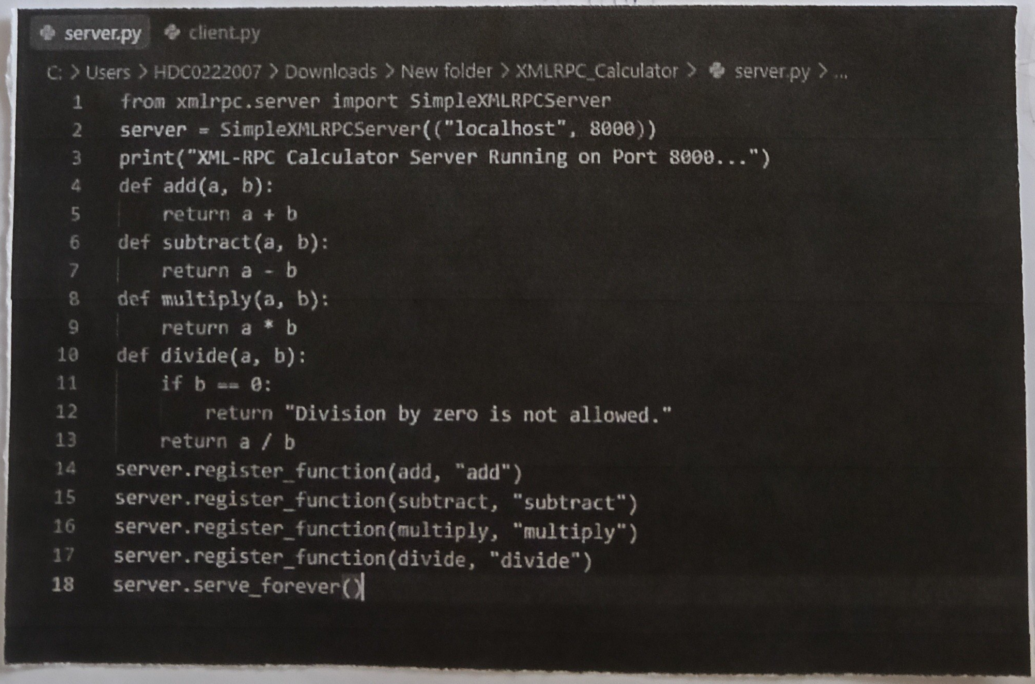
4) Display FTP server welcome message.

5) Attempt anonymous login → If login successful display message anonymous access permitted.

6) use nlst() to check whether the directory contents are accessible.


server.py
client.py
C: > Users > HDC0222007 > Downloads > New folder > XMLRPC_Calculator > client.py > ...

```python
import xmlrpc.client
server = xmlrpc.client.ServerProxy("http://localhost:8000")
a = int(input("Enter First Number: "))
b = int(input("Enter Second Number: "))
print("\nSelect Operation")
print("1. Addition")
print("2. Subtraction")
print("3. Multiplication")
print("4. Division")
choice = input("Enter your choice (1-4): ")
if choice == '1':
    print("Result =", server.add(a, b))
elif choice == '2':
    print("Result =", server.subtract(a, b))
elif choice == '3':
    print("Result =", server.multiply(a, b))
elif choice == '4':
    print("Result =", server.divide(a, b))
else:
    print("Invalid Choice")
```


server.py
client.py
C: > Users > HDC0222007 > Downloads > New folder > XMLRPC_Calculator > server.py > ...

```python
from xmlrpc.server import SimpleXMLRPCServer
server = SimpleXMLRPCServer(("localhost", 8000))
print("XML-RPC Calculator Server Running on Port 8000...")
def add(a, b):
    return a + b
def subtract(a, b):
    return a - b
def multiply(a, b):
    return a * b
def divide(a, b):
    if b == 0:
        return "Division by zero is not allowed."
    return a / b
server.register_function(add, "add")
server.register_function(subtract, "subtract")
server.register_function(multiply, "multiply")
server.register_function(divide, "divide")
server.serve_forever()
```

EX : 6

AIM:

To devel

scanner an

determine

anonymous

Procedure:

1) Instal

class fro

2) Define

3) Ente

establish

port 2

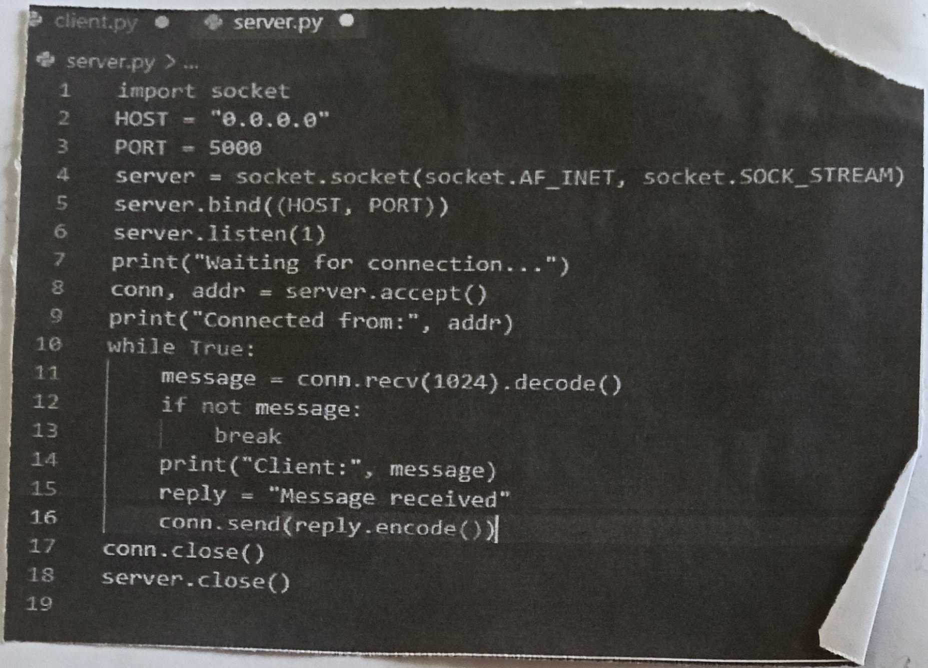
4) Display

5) Atte

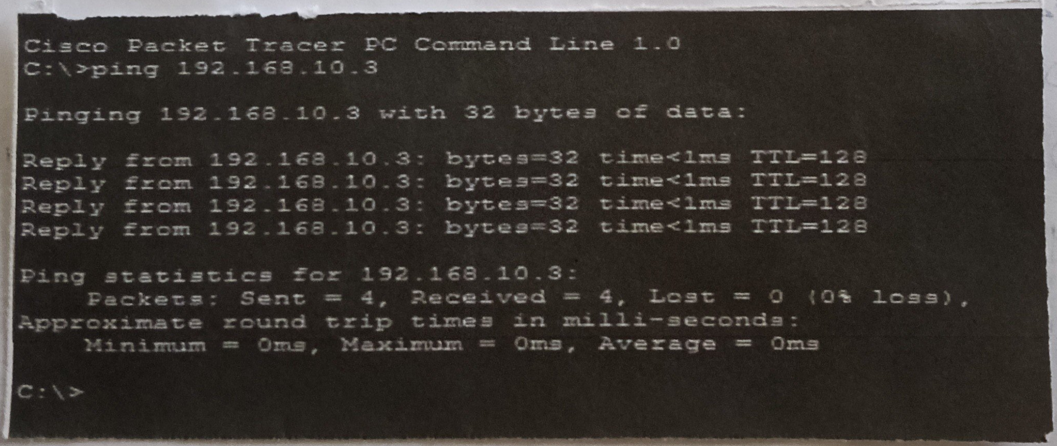
display

6) ue

contents



```python
import socket
HOST = "0.0.0.0"
PORT = 5000
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen(1)
print("Waiting for connection...")
conn, addr = server.accept()
print("Connected from:", addr)
while True:
    message = conn.recv(1024).decode()
    if not message:
        break
    print("Client:", message)
    reply = "Message received"
    conn.send(reply.encode())
conn.close()
server.close()
```



```
Cisco Packet Tracer PC Command Line 1.0
C:\>ping 192.168.10.3

Pinging 192.168.10.3 with 32 bytes of data:

Reply from 192.168.10.3: bytes=32 time<1ms TTL=128
Reply from 192.168.10.3: bytes=32 time<1ms TTL=128
Reply from 192.168.10.3: bytes=32 time<1ms TTL=128
Reply from 192.168.10.3: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.10.3:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\>
```

Result :

Thus an

implemen

Result :

Thus an anonymous FTP scanner was successfully implemented using ftplib